NFTsim User Guide

`NFTsim` is a `C++` program developed by the Brain Dynamics Group at the University of Sydney, which solves the neural field model of Robinson et al. It is one of a core group of software analysis packages we have developed for our research.

The role of `NFTsim` is to integrate neural field equations to predict neural activity in neural field models with arbitrary numbers of populations and connections between them. As such, `NFTsim` can be viewed as a generalized neural field integrator that enables users to:

1. Specify an arbitrary population model: an arbitrary number of populations and connections them may be specified;
2. Choose different types of populations, including neural or stimulus populations. For each neural population, the type of firing response, dendritic response type may be specified.
3. Choose different types of connections, including the type of axonal propagation, and synaptic coupling.
4. For each object, specify the parameter values.

This user guide covers how to obtain and set-up `NFTsim` (Sec. 1), how to configure (Sec. 2.1) and launch a simulation (Sec. 2), as well as some possible postprocessing of the output (Sec. 3). It concludes with a section on development, that is, things you need to know if you intend to extend the core functionality of `NFTsim` (Sec. 4).

Within this documentation, specific terminology as appeared in the computer is in `typewriter font` . Commands are denoted as

```
Command to put in computer
```

Assumed knowledge

To get the most out of `NFTsim` , you will require a working knowledge of neural field theory. In `NFTsim` , each neural field component is handled by an object:

$$P = \nu_{ab}\phi_{ab}, \qquad \texttt{Coupling}$$
$$D_{ab}V_{ab} = P, \qquad \texttt{Dendrite}$$
$$Q_a = S_a\big[\sum_b V_{ab}\big], \qquad \texttt{FiringResponse}$$
$$\mathcal{D}_{ab}\phi_{ab} = Q_b. \qquad \texttt{Propagator}$$

A detailed understanding of these equations is not essential to use `NFTsim` . However, you should be familiar with the roles of these components in neural field models. TODO - GO HERE TO FIND OUT MORE

© Brain Dynamics Group, University of Sydney 2015

NFTsim input and output files are both written in plain text, with syntax detailed in this manual. The analysis routines included with NFTsim are written in Matlab, and this documentation assumes that you are already familiar with basic use of Matlab. TODO - GO HERE TO FIND OUT MORE

1 Obtaining and setting up NFTsim

1.1 Obtaining NFTsim

NFTsim can be obtained from our website, http://sydney.edu.au/science/physics/research/complex-systems/brain-dynamics/software.shtml. As per the license agreement, users are not permitted to redistribute this program to other individuals or organizations - they should download NFTsim directly so as to have the most recent version, including any corrections.

1.2 Directory layout

All of the files associated with NFTsim are contained in a directory called nftsim . This directory is referred to as the 'root directory', and it contains the following:

src/	C++ source code.
(obj/)	This directory is created during compilation and stores intermediate object code.
(dep/)	This directory is created during compilation and stores dependency information.
(bin/)	The compiled binary nftsim is created here.
configs/	Stores example configuration files for NFTsim .
doc/	Contains this manual and the reference manual once it's built.
+nf/	Matlab package directory containing NFTsim helper scripts.
test/	Directory for development testing and is irrelevant for users.

1.3 Compiling NFTsim

You can compile NFTsim simply by running

```
make
```

from the root directory. The compiler command is specified in Makefile . On Linux this defaults to g++ and on Mac it defaults to clang++ . To run NFTsim on Windows, we suggest cross-compiling using MinGW on a Unix-like system. Example compiler flags for this are included in Makefile . Compiling with the Microsoft Visual C++ compiler has not been tested.

The Makefile also provides a number of additional commands. This documentation along with

© Brain Dynamics Group, University of Sydney 2015

the reference-manual can be generated by running

```
make docs
```

which will recompile the LaTeX files in the `doc/` directory if they've been updated since the last time this .pdf was generated, and run doxygen to generate the reference-manual from the source code. The reference manual can then be viewed by pointing your web-browser at

```
doc/html/index.html
```

To delete the dependency files, object code, and temporary LaTeX files (e.g., to perform a clean compile of the program), you can use

```
make clean
```

which will delete the `dep/*.d` and `obj/*.o` files, as well as the temporary LaTeX files in `doc/`. For a list of additional make targets, including brief descriptions of their function, run

```
make help
```

2 Running NFTsim

You can run `NFTsim` directly using

```
./bin/nftsim
```

from the main `NFTsim` directory. For ease of use, you may consider adding the `nftsim` binary to your system path.

By default, `NFTsim` will check if a configuration file called `nftsim.conf` exists in the current directory. If this file exists, `NFTsim` will run it and write output to the file `nftsim.output` in the current directory. When these default files are used, a warning is displayed:

```
romesha@romeshalt: nftsim > ./bin/nftsim
Warning: Using nftsim.conf for input by default
Warning: Using nftsim.output for output by default
```

Note that NFTsim will overwrite the output file if it exists, so it is not a good idea to use `nftsim.output` for your work.

You can optionally provide an input file using the `-i` switch. For example,

```
./bin/nftsim -i configs/example.conf
```

will use the configuration file `example.conf` within the `configs` directory, and will write output

© Brain Dynamics Group, University of Sydney 2015

to `example.output` also in the `configs` directory. The output file name is generated by replacing the input file's suffix (`.conf`) with `.output` .

Or you can also specify an output file, using the `-o` switch. For example, if you want your configuration files and output data to be in different directories,

```
./bin/nftsim -i configs/example.conf -o example.output
```

will use the configuration file `example.conf` within the `configs` directory, and will write output to `example.output` in the current directory.

Output is written as text by default. For large outputs (many nodes, or a short output interval) a binary format is much faster to write and to read, and about a third of the size:

```
./bin/nftsim -i configs/example.conf -o example.output --binary
```

`--format Binary32` writes single-precision values instead, half the size again, and `--format Text` the default text. The format can also be set in the configuration file (`Format:` , Sec. 2.1.6); the command line takes precedence. The binary format is described in Sec. 3.1.

A list of available switches can be displayed by using the `-h` or `--help` option i.e., `./bin/nftsim -h`

2.1 Writing a configuration file

`NFTsim` allows an arbitrary number of populations and connections between them, with all objects taking arbitrary parameter values. These are all configured via a configuration file. This section documents the specifications of configuration files, where we use `configs/example.conf` as an illustrative example.

To write a configuration file, follow these steps:

1. Determine your population model by drawing a schematic diagram, thereby constructing a connection matrix. An example is shown in Fig. 1.
2. Look up existing configuration files in `configs/` . By checking the comment located at the top of a configuration file, and also the connection matrix, a user should find the most suitable existing file to construct his own. This is less tedious (and less error prone) than writing a new one from scratch.
3. Specify the global parameters and connectivity matrix (Sec. 2.1.2).
4. Specify all populations (Sec. 2.1.3).
5. Specify all propagators (Sec. 2.1.4).
6. Specify all couplings (Sec. 4.2.4).

© Brain Dynamics Group, University of Sydney 2015

7. Specify all output requests (Sec. 2.1.6).

General rules on the entries within a configuration file:

1. The structure of a configuration file is: 1) comment 2) global information 3) object specification 4) output specification.
2. Object specification involves first specifying the object type. The syntax is the object identifier, followed by its type, then a hyphen:

   ```
   Object 1: Type -
   ```

   Then the object parameters are specified, following this pattern:

   ```
   Object parameter: value
   ```

3. Most parameters are essential. Failure to provide these parameters would result in `NFTsim` terminating with an error message. A minority of the parameters are optional.
4. The ordering of the parameters are important. Wrong parameter ordering results in `NFTsim` terminating with an error message.
5. The configuration file is white-space independent, e.g., there can be either no spaces, many spaces, or new lines between parameters.
6. Object labels must be written exactly as `Population 1:` , `Propagator 1:` , `Coupling 1:` and `Dendrite 1:` (one space before the number, and the colon straight after it). Each label starts a section that runs to the next label (`Output:` starts the output specification), and an object's parameters are read only from its own section, so that, e.g., a stimulus population without a `Mean` starts from zero rather than from another population's `Mean` .
7. For readability, users are encouraged to arrange parameter entries for different objects (via new lines and indentations) and aligning corresponding parameters between different objects.
8. Tip for `vi` users: `./Helper_scripts/nftsim.vim` implements syntax highlighting for configuration files in `vi` . See comments within for installation instructions.

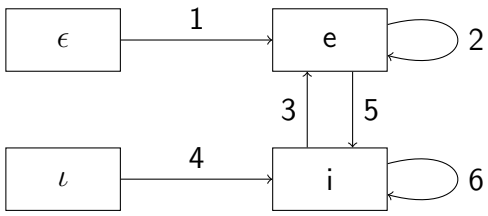
2.1.1 Example configuration

The remainder of this section refers to an example configuration for the illustrative example system shown in Fig. 1. The corresponding configuration file is also shown below. This configuration file is included in the repository as `configs/cortex.conf` and can be run from the `NFTsim` repository root using

```
./bin/nftsim -i configs/cortex.conf -o cortex.output
```

It is recommended that you open `configs/cortex.conf` *to read in conjunction with this section of the manual.*

© Brain Dynamics Group, University of Sydney 2015



From:	ϵ	ι	e	i
To ϵ:	0	0	0	0
To ι:	0	0	0	0
To e:	1	0	2	3
To i:	0	4	5	6

Figure 1: Top: schematic diagram of a purely cortical population model comprising excitatory and inhibitory populations, as well as two stimulus populations; each arrow indicates a connection between populations, so that each stimulus connects to a cortical population, and each cortical population connects to all cortical populations. Bottom: connection matrix indicating the connections between populations; zero indicates no connection, and a connection is indicated by a nonzero number, ordered top to bottom, left to right.

© Brain Dynamics Group, University of Sydney 2015

2.1.2 Global information

- *Initial comments*

```
Example config file of cortical model with excitatory and
   inhibitory neurons
```

Any text before the text `Time:` , is disregarded by `NFTsim` and serves as comment, which is strongly recommended for all configuration files. The comment may contain any text, including colons and object labels such as `Population 2` , except the text `Time:` itself.

- *Integration time and timestep*

```
Time: 10 Deltat: 1e-4
```

`Time` is the simulation duration in seconds.

`Deltat` is the time increment for each time step.

- *Time-stepping scheme (optional)*

```
Integration: SecondOrder
```

`Integration` is optional and, if given, must directly follow `Deltat` . Its value is `Default` (the same as leaving it out) or `SecondOrder` .

With the default scheme each step updates the couplings (using ϕ_{ab} at t_n), then the dendrites, holding $\nu_{ab}\phi_{ab}$ at its value at t_n over $[t_n, t_{n+1}]$ (or, for `Integral` dendrites, taking it to rise linearly from its value at t_{n-1} to its value at t_n, which lags the input by one step), then the firing rates $Q_a(t_{n+1}) = S[V_a(t_{n+1})]$, then the propagators. Delays are rounded to a whole number of steps. The error of the coupled system is then first order in `Deltat` .

With `SecondOrder` , each dendrite takes $\nu_{ab}\phi_{ab}$ as linear over $[t_n, t_{n+1}]$, extrapolated from its values at t_{n-1} and t_n (as in a second-order Adams–Bashforth step); the `Harmonic` propagator takes the delayed Q_b as linear between its values at t_n and t_{n+1}, which are both known when propagators step (`HarmonicIntegral` and `Wave` already use them); and a delay with $\tau_{ab}/\Delta t = m + f$, $0 < f < 1$, is evaluated by linear interpolation, $Q_b(t - \tau_{ab}) \approx (1 - f)\, Q_b(t - m\Delta t) + f\, Q_b(t - (m + 1)\Delta t)$, rather than rounded. Each of these has an $O(\Delta t^3)$ error per step, so the coupled system, including the nonlinear firing response, is second order: halving `Deltat` reduces the error about four-fold. The scheme is explicit and costs about the same as the default. A fixed point of the model is also a fixed point of the scheme.

Second order requires smooth inputs: stimuli with jumps (`PulseRect` , `Burst`) give first-order errors near each jump with either scheme, and white noise converges in its statistics rather than path by path. Modules that integrate their own state with their inputs held constant over a step remain first order: the plastic couplings (`CaDP` , `BCM` , `BCM-Spatial`), the `Bursting` and `GlutamateResponse` firing responses, and `WaveLegacy` . The `Ramp` and `DiffArctan` couplings evaluate ν_{ab} one step ahead of ϕ_{ab}, which is first order while ν_{ab} changes, and `Ramp` dendrites hold their rates constant within a step (at its midpoint). The spatial error of `Wave` depends on Δx and is unaffected.

- *Grid size*

© Brain Dynamics Group, University of Sydney 2015

```
Nodes: 4 Longside: 2
```

`Nodes` is the number of grid points in the spatial dimension per population of neurons. The code has been explicitly designed to have equal number of neurons per population.

`Longside` is an optional parameter, specifying the longside of the rectangular grid. If it is not supplied, it is assumed to be a square.

Both spatial dimensions have periodic boundary conditions, so that populations have the topology of a torus.

Note that the physical size of each neural population is specified in the definition of the population - see Sec. 2.1.3. For each population, Δx is automatically computed based on the number of grid points and the physical size of the population. This ensures that the physical size of the system does not change when the number of grid points is changed. When a wave propagator is included, the value of Δx is automatically selected from the presynaptic population.

- *Specifying connectivity*

```
Connection matrix:
From: 1 2 3 4
To 1: 0 0 0 0
To 2: 0 0 0 0
To 3: 1 0 2 3
To 4: 0 4 5 6
```

We next specify a square connection matrix, where each entry is the connection from the column population to the row population. Zero indicates no connection, while a nonzero number indicates a connection. This number must be indexed from top to bottom, left to right.

The size of this matrix determines the number of neural populations in the simulation. The number of nonzero connections determines the number of dendrites, couplings and propagators that will be present in the configuration file.

2.1.3 Population data

This section contains population information sections. There are two types of neural populations: ordinary populations and stimulus populations:

Stimulus populations

`NFTsim` identifies stimulus populations as populations which have no dendrites, i.e., the row for that population contains no nonzero elements. Each stimulus population information section is as follows.

- ```
  Population 1: Stimulation
  ```

© Brain Dynamics Group, University of Sydney 2015
  ```

The identifier `Population 1` is required for cross-checking.

The descriptor `Stimulation` is not parsed by `NFTsim` , but it is strongly recommended for human referencing.

- ```
  Length: .5
  ```

The physical edge length of the population (which is a 2D sheet), in metres. This is used in `Wave` propagators and the `ASD` of `White` stimulus.

- ```
  Stimulus: Const - Onset: 0
  ```

The identifier `Stimulus` is required for cross-checking.

This is followed by the type of stimulus, to be further elaborated below.

Optional parameter `Onset` specifies the time [s] from the start of the simulation for the stimulus to begin. If unspecified, stimulus starts at time 0.

Optional parameter `Duration` can be used to specify the length of time [s] that the stimulus is active. If unspecified the stimulus continues for the entire simulation.

If optional parameter `Node` is specified, only the specified node indices will receive stimulation.

Possible stimulus patterns:

Constant

```
Const - Mean: 5
```

PulseRect

```
PulseRect - Amplitude: 1 Width: 2e-2 Frequency: 1 Pulses: 1
```

PulseSigmoid

```
PulseSigmoid - Amplitude: 1 Width: 2e-2 Frequency: 1
   Pulses: 1
```

Sine

```
Sine - Amplitude: 1.0 Frequency: 2.0
```

White

Gaussian noise, characterized by the mean and standard deviation:

```
White - Mean: 1 StdDev: 20 Ranseed: 10
```

Alternatively, the amplitude spectral density (ASD), which is the square root of the power spectral density (PSD) may be specified instead of the standard deviation (StdDev). The advantage of using ASD is that NFTsim will preserve the ASD under changes in `Deltat` , population `Length` and spatial `Nodes` . Given the ASD, `NFTsim` calculates the standard deviation:

© Brain Dynamics Group, University of Sydney 2015

```
White - Mean: 1 ASD: 20 Ranseed: 10
```

In general, it is preferable to specify the noise amplitude using `ASD` rather than `StdDev`.

Convention. `ASD` ^{2} is the *two-sided* power spectral density of the noise with respect to *angular* frequency ω, normalised so that the variance is its integral over ω from $-\pi/\Delta t$ to $\pi/\Delta t$. For a single node this gives

$$\sigma = \sqrt{\frac{2\pi\,\mathrm{ASD}^2}{\Delta t}},$$

and on a grid, where each spatial Fourier mode is also included,

$$\sigma = \sqrt{\frac{2\,(2\pi)^2\pi\,\mathrm{ASD}^2}{\Delta t\,\Delta x^2}} = \sqrt{\frac{8\pi^3\,\mathrm{ASD}^2}{\Delta t\,\Delta x^2}},$$

with $\Delta x =$ `Length` $/$`Longside nodes` (or `Length` $/\sqrt{}$`Nodes` on a square grid), the same spacing as the `Wave` propagator.

The note `doc/noise.pdf` (2012) derives the same expressions without the leading factor of 2, because it treats $\phi_n^2(\omega)$ as a *one-sided* spectral density. The factor of 2 was added in 2018 (commit `a2d4a46`) so that `ASD` refers to the two-sided density, and the shipped configuration files were rescaled to match (for example, `ASD: 7.071067811865475e-06` $= 10^{-5}/\sqrt{2}$). To reproduce a result that used the one-sided convention, divide its `ASD` by $\sqrt{2}$.

If spectra are expressed per unit linear frequency f instead of ω, the power spectral densities differ by a factor of 2π (since $d\omega = 2\pi\,df$), so $\mathrm{ASD}(f) = \sqrt{2\pi}\,\mathrm{ASD}(\omega)$.

The random number generator may be specified in `Ranseed` . If a seed is not specified, an automatically-incremented seed will be used instead, so that multiple stimulus populations will have independent sequences. In general it is not necessary to set the seed manually unless different random numbers are required for otherwise identical runs. Because the automatic seeds are always 1, 2, 3, ..., repeated runs of the same configuration produce identical noise; set `Ranseed` explicitly to obtain independent realisations.

Superimposing stimuli

To superimpose 2 or more stimuli, begin with the keyword `Superimpose` , followed by the number of stimuli. Then list all the stimulus patterns and their parameters, with each stimulus pattern preceded by the keyword `Stimulus` .

```
Stimulus: Superimpose: 2
 Stimulus: White - Mean: 1 ASD: 1
 Stimulus: PulseRect - Onset: 0.5 Width: 2e-2 Frequency 1
    Pulses: 1
```

Example stimuli-only.conf

```
Time: 12 Deltat: 9.765625e-04
Nodes: 144

    Connection matrix:
```

```
From:  1
To 1:  0


Population 1: Stimulation
Length: 0.5
 Stimulus: Superimpose: 2
 Stimulus: PulseRect - Onset: 2 Node: 62 Amplitude: 1.0
    Width: 1.0 Period: 3 Pulses: 3
 Stimulus: PulseSigmoid - Onset: 2 Node: 42 Amplitude: 1.0
    Width: 1.0 Period: 3 Pulses: 3 Sigma: 0.03125

Output: Node: 33 42 62 77 Start: 0
Population: 1
Dendrite:
Propagator:
Coupling:
```

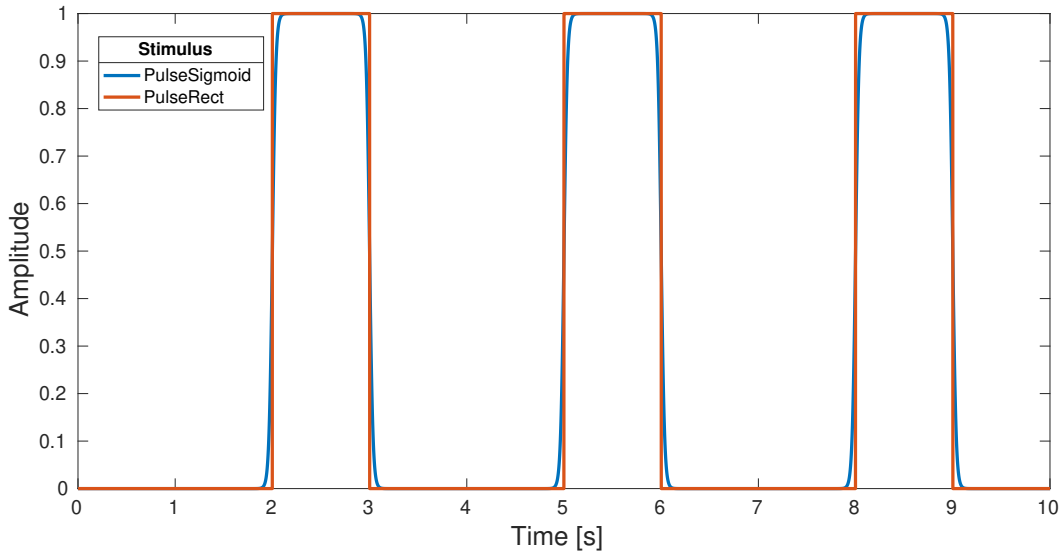


Figure 2: Note than in this example, the parameter `Onset` of `PulseSigmoid` is the time at which the sigmoid pulse reaches half of its range.

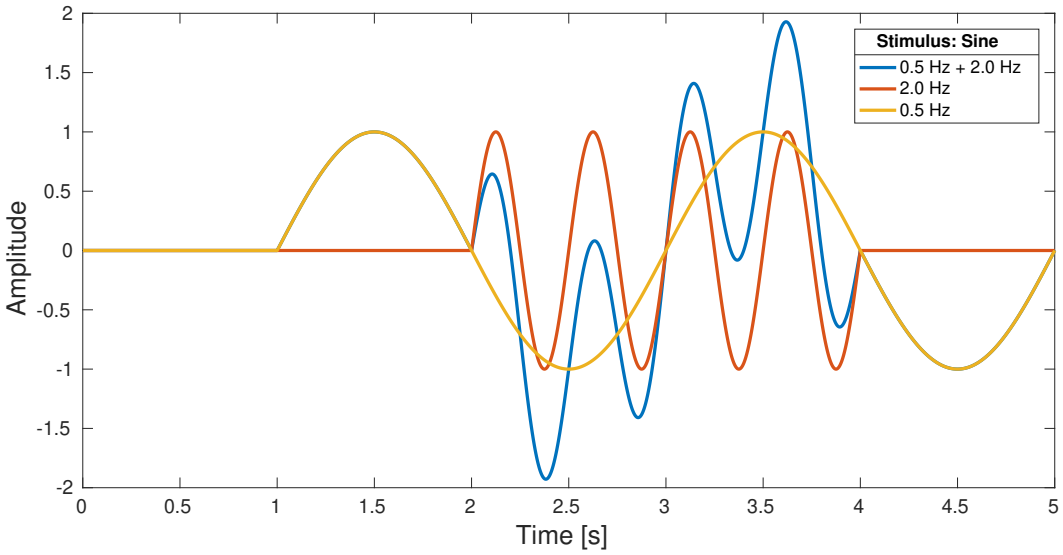
```
Time: 5 Deltat: 9.765625e-04
Nodes: 144

    Connection matrix:
From:  1
To 1:  0


Population 1: Stimulation
Length: 0.5
 Stimulus: Superimpose: 4
 Stimulus: Sine - Onset: 1 Duration: 4 Node: 62 Amplitude:
```

```
    1.0 Frequency: 0.5
 Stimulus: Sine - Onset: 2 Duration: 2 Node: 42 Amplitude:
    1.0 Frequency: 2.0
 Stimulus: Sine - Onset: 1 Duration: 4 Node: 33 Amplitude:
    1.0 Frequency: 0.5
 Stimulus: Sine - Onset: 2 Duration: 2 Node: 33 Amplitude:
    1.0 Frequency: 2.0

Output: Node: 33 42 62 Start: 0
Population: 1
Dendrite:
Propagator:
Coupling:
```



Ordinary populations

Any non-stimulus population is an ordinary population.

- ```
  Population 3: Excitory neurons
  ```

  The identifier `Population 3` is required for cross-checking.

  The descriptor `Excitatory neurons` is not parsed by `NFTsim` , but it is strongly recommended for human referencing.

- ```
  Q: 8.87145
  ```

  The initial firing rate. `NFTsim` starts the simulation as if the model were at a steady state with these firing rates: by default every propagator starts at $\phi_{ab} = Q_b$ and every dendrite at $V_{ab} = \nu_{ab}\phi_{ab}$. This is a steady state only if each neural population's `Q` equals its firing response to that input, $Q_a = S_a(V_a)$ with $V_a = \sum_b \nu_{ab}\phi_{ab}$. Otherwise the firing rate jumps to $S_a(V_a)$ in the first time-step, and the simulation starts with a transient. When it starts, `NFTsim` checks this for each neural population, node by node, with the initial $\nu_{ab}\phi_{ab}$ of each coupling (for plastic and time-varying couplings,

© Brain Dynamics Group, University of Sydney 2015

their initial ν), and prints a warning to the standard error if $S_a(V_a)$ and `Q` differ by more than 0.1% (relative), for example

```
WARNING: Population 2's initial Q (8.87145) is not a
   steady state: its initial inputs give Q = 11.0797 (soma
   potential 1.153e-04 V). The simulation will start with
   a transient.
    Newton's method from the configured values converges
       to this steady state (its stability is not checked):
      Population 2: Q: 11.16405416
```

It also warns if a propagator's initial `phi` (Sec. 2.1.4) differs from its presynaptic population's `Q` . After any warning, `NFTsim` solves the steady-state equations $Q_a = S_a(\sum_b \nu_{ab} Q_b)$ for the whole network by Newton's method, starting from the configured `Q` , and suggests the firing rates it finds as `Q` values; stimulus populations keep their initial firing rate. It makes no suggestion if Newton's method does not converge, or if the network is not spatially uniform (eg, `nu` or `phi` given per node), and the suggested steady state may be unstable, or one of several. The check is diagnostic only: it does not change the simulation or its output. It skips stimulus populations and `Bursting` populations, whose steady-state firing also depends on their slow threshold variables.

- ```
  Firing: Function: Sigmoid Theta: 13e-3 Sigma: 3.8e-3 Qmax:
     340
  ```

  Specify the sigmoidal firing response of the population,

  $$Q = \frac{Q_{\max}}{1 + \exp[-(V - \theta)/\sigma']}.$$

  `Sigma` is σ' in this formula (Eq. 10 of Sanz-Leon et al., 2018). The standard deviation of firing thresholds across the population is $\sigma = \sigma'\pi/\sqrt{3}$, so `Sigma` is that standard deviation already divided by $\pi/\sqrt{3}$. (Table 1 of the 2018 paper describes σ' itself as the standard deviation; the text under its Eq. 10, and the code, use the definition given here.)

  Alternatively, you can specify a linear, quadratic or cubic firing response,

  ```
  Firing: Function: Linear a: 1 b: 1
  Firing: Function: Quadratic a: 1 b: 1 c: 0
  Firing: Function: Cubic a: 1 b: 1 c: 0 d: 0
  ```

  which give $Q = aV + b$, $Q = aV^2 + bV + c$ and $Q = aV^3 + bV^2 + cV + d$, respectively. Any other `Function` name is an error.

- ```
  Dendrite 1: V: Steady alpha: 83 beta: 769
  ```

  The identifier `Dendrite 1` , where the number 1 is the presynaptic connection index, is required for cross-checking. Users should find that these indices are simply ordered as 1, 2, 3, 4, ...

  Optional parameter `V` may be used to specify the initial depolarization contribution from presynaptic activity. If unspecified, or set to `Steady` , `NFTsim` sets it to the coupling's initial input $\nu_{ab}\phi_{ab}$ at each node, using the initial ϕ_{ab} of the presynaptic propagator (its `phi` , Sec. 2.1.4). This is the input the coupling actually delivers: with

© Brain Dynamics Group, University of Sydney 2015

one `nu` per node it uses each node's value, and for a `Matrix` coupling it is the sum over source nodes, $\sum_i \nu_{ij}\phi_i$.

`alpha` and `beta` are the parameters for the depolarization response.

2.1.4 Propagation data

- `Propagator 1:`

  This identifier is required for cross-checking.

- A propagator type is required at this point. Choices are `Map` , `Wave` , `WaveLegacy` , `Harmonic` and `HarmonicIntegral` .

  Map

  `Map - phi: Steady Tau: 0`

  This propagator is the mapping propagator where spatial spreading is negligible. Its form is given by

  $$\phi_{ab}(\mathbf{r}, t) = Q_b(\mathbf{r}, t - \tau_{ab}).$$

  Optional parameter, `Tau` , is the axonal delay term. If unspecified, it is taken as zero. Since all propagators contain this object, its description is given below.

  Optional parameter `phi` is the initial axonal firing rate, described below. For `Map` it sets ϕ_{ab} at $t = 0$ only, which the couplings use in the first time-step; from then on ϕ_{ab} is the delayed Q_b.

  Wave

  This propagator is the wave equation propagator governed by the equation

  $$\left[\frac{1}{\gamma_{ab}^2}\frac{d^2}{dt^2} + \frac{2}{\gamma_{ab}}\frac{d}{dt} + 1 - r_{ab}^2\nabla^2\right]\phi_{ab}(\mathbf{r}, t) = Q_b(\mathbf{r}, t - \tau_{ab}).$$

  `NFTsim` checks whether the Courant condition must be satisfied, i.e.

  $$\Delta t/\Delta x < \sqrt{2}/r_e\gamma_e,$$

  where Δx is the population length per node.

  The propagator input is given by

  `Wave - phi: Steady Tau: 0 Range: 80e-3 gamma: 116`

  Optional parameter `phi` is the initial axonal firing rate, described below. The wave equation starts with $\partial\phi_{ab}/\partial t = 0$ at every node.

  `Range` is r_{ab} in the wave equation.

  `gamma` is the damping coefficient. Alternatively, `velocity` may be specified, and `gamma` is calculated via $\gamma_{ab} = \mathrm{v}_{ab}/r_{ab}$.

  In case there is only one node, this degenerates into a `Harmonic` propagator.

  `WaveLegacy` takes the same parameters and solves the same equation with an older nine-point stencil, kept to reproduce published results.

© Brain Dynamics Group, University of Sydney 2015

Harmonic This is a harmonic oscillator implementation of the damped wave equation, with no spatial variations:

$$\left[\frac{1}{\gamma_{ab}^2}\frac{d^2}{dt^2} + \frac{2}{\gamma_{ab}}\frac{d}{dt} + 1\right]\phi_{ab}(\mathbf{r},t) = Q_b(\mathbf{r}, t-\tau_{ab}).$$

The input form is given by

```
Harmonic - phi: Steady Tau: 0 gamma: 116
```

It starts from the initial `phi` at rest ($d\phi_{ab}/dt = 0$). `HarmonicIntegral` takes the same parameters, and steps the same equation with its exact solution for Q_b varying linearly over each time-step. It also starts at rest, taking the delayed Q_b before $t = 0$ to equal the initial ϕ_{ab} at each node, so that an explicit `phi` is treated as the steady state of an earlier constant input.

phi

The initial axonal firing rate ϕ_{ab}, which must be the first entry after the propagator type. If unspecified, or set to `Steady` , it is the initial firing rate of the presynaptic population, $\phi_{ab} = Q_b$, at every node. Otherwise, as for `Tau` and `nu` , it is either one number, used at every node, or n numbers, one per node, where $n =$ `Nodes` ; any other number of values is an error. The per-node form sets arbitrary initial conditions. For example, with `Nodes: 4` ,

```
Propagator 1: Harmonic - phi: 1 1 9 1 Tau: 0 gamma: 116
```

starts ϕ_{ab} at 9 at node 3 and at 1 elsewhere. The values may span several lines. `test/numerical/wave-initial-phi.conf` starts a `Wave` propagator from a localized bump. A dendrite whose `V` is `Steady` starts at $\nu_{ab}\phi_{ab}$ with this initial ϕ_{ab}, node by node.

Tau

The axonal time delay between populations. If it is spatially homogeneous, then it is a number with units of seconds. If it is spatially inhomogeneous, then input n numbers, where $n =$ `Nodes` . With the default time-stepping scheme, delays are rounded to the nearest whole number of time-steps (an error of up to $\Delta t/2$ in the delay), and a warning is printed if a value is not within 10^{-6} of a step multiple. Choosing `Deltat` as a power of two (e.g. 2^{-13} s) lets common delays be represented exactly. With `Integration: SecondOrder` (Sec. 2.1.2) a fractional delay is instead interpolated linearly between the two nearest stored time points.

2.1.5 Coupling Classes

- The type of coupling between each pair of populations can be specified for separately. The coupling data is given as n lines corresponding to each nonzero connection in the connection matrix defined at the top of the file. Different types of coupling has their own input form.
- ```
  Coupling 1:
  ```
- This syntax serves the purpose of an identifier for cross-checking. The following types of coupling are available: `Map` , `Matrix` ,`Ramp` , `CaDP` , `BCM` and `DiffArctan` .

© Brain Dynamics Group, University of Sydney 2015
  ```

Map

Linear synaptic coupling with input parameter `nu` , constant time. This parameter may be a single number or take a vector of length `Nodes`, which allows for the expression of nonuniform models.

```
        Map - nu: 0.0012
```

where `nu` is the synaptic coupling parameter. In NFT, it corresponds to the product of the mean synaptic strength s_{ab} and N_{ab}, the mean number of connections from cells of type b to cells of type a.

Ramp

Piecewise linear segments of increasing or decreasing `nu` can be specified by passing an array of nu values `nu` and array of time points `timepoints` , which together they specify the breakpoints of a linear segment. Also, this coupling type assumes that at `t=0` , `nu=nu[0]` , ie, the segment between `t=0` and timepoints[0] is constant. This behavior is appropriate to let the system reach a steady-state or any other known stable attractor.

```
        Coupling 1:  Ramp - nu: 0.001525377176 0.001527
           0.001527 0.00153 0.00153 0.001525377176
           timepoints: 5 10 15 17.5 19.5 20
```

The resulting profile can be seen in Fig. 2.1.5.

Matrix

The coupling may be expressed as a connection matrix, where the coupling strengths do not change with time. The format of the `nu` matrix is the same as the population connection matrix, each row is to the same node, each column is from the same node. When outputting, each specified outputting node output the indexed row.

```
nu:
  13e-6 0
  0 13e-6
```

CaDP

Calcium dependent plasticity according to Fung and Robinson.

```
CaDP - nu: 13e-6 nu_max: 80e-6 Dth: .25e-6 Pth: .45e-6
   xyth: 1e-4 x: 2.3e-2 y: 2e-2 B: 30e3 glu_0: 200e-6
   gNMDA: 2e-3
```

`Dth` and `Pth` are the calcium-plasticity thresholds; `xyth` , `x` and `y` are the plasticity rates; `B` , `glu_0` and `gNMDA` are NMDA receptor parameters.

To use `CaDP` , glutamate dynamics must be specified for the postsynaptic population. In the end of the relevant population entry, append

```
Glutamate dynamics - Lambda: 150e-6 tGlu: 30e-3
```

`Lambda` is the glutamate concentration rise per presynaptic spike; `tGlu` is the decay timescale for glutamate dynamics.

BCM

© Brain Dynamics Group, University of Sydney 2015

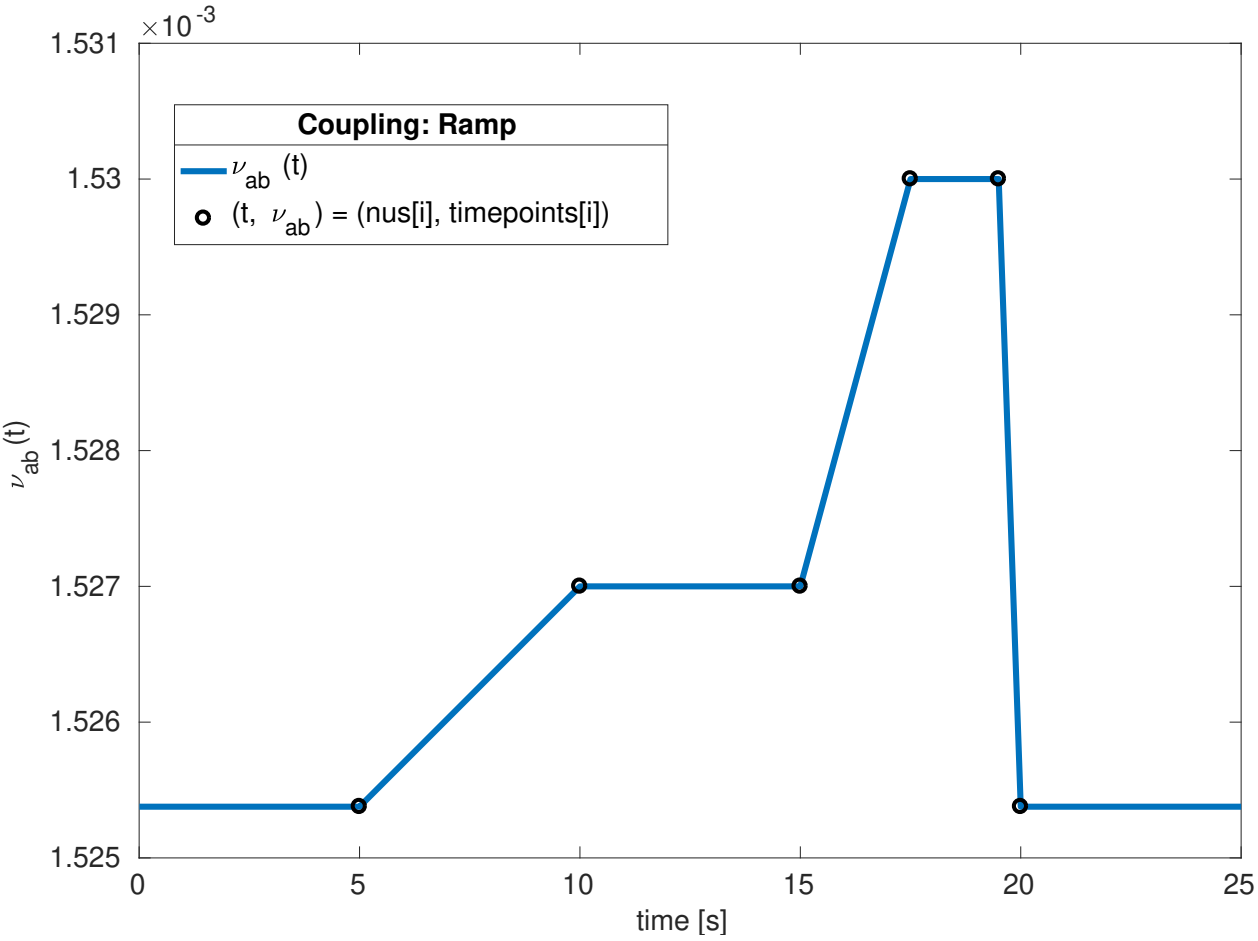


Figure 3: The resulting temporal profile of a ν_{ab} defined with `Ramp` . The circles highlight the breakpoints (`nu` and `timepoints`) specified in the configuration file.

Extends `CaDP` with metaplasticity according to Fung and Robinson. it has an additional parameter `t_BCM` , the timescale of metaplasticity.

```
CaDP - nu: 13e-6 nu_max: 80e-6 Dth: .25e-6 Pth: .45e-6
   xyth: 1e-4 x: 2.3e-2 y: 2e-2 B: 30e3 glu_0: 200e-6
   gNMDA: 2e-3 t_BCM: 7
```

DiffArctan

This type of coupling allows the connection strength ν_{ab} to vary as a function of time, such that ν can be smoothly ramped up and down.

```
DiffArctan - nu_min: 0.002  nu_max: 0.006  delta: 10
   t_half_up:50  t_half_down:100
```

where `nu_min` and `nu_max` are the min and max ν's respectively. `delta` determines the slope of the ramp ($1/$`delta`) and is the time interval in which the connection strength increases (decreases) from 0.25 to 0.75 of `nu_max` . `t_half_up` and `t_half_down` are the half way times to maximum and minimum amplitude, respectively.

© Brain Dynamics Group, University of Sydney 2015

2.1.6 Output data

NFTsim outputs field quantities (i.e. a neurodynamic quantity which takes a value for each node) with respect to nodes and time. By default, the output file is generated by replacing the input configuration file's suffix (`.conf`) with `.output`. The output file can be changed by launching the program with the `-o` switch.

- ```
  Output:
  ```

  Begin with the `Output` declaration.

- ```
  Node: 1 2
  ```

  Enumerate all nodes to be outputted. If outputting all nodes, use shorthand `All`. If no nodes are specified, no nodes will be outputted.

- ```
  Start: 0 Interval: 1e-4
  ```

  Optional parameters for the time to start output, and optional parameter for time interval between outputs.

  If undefined, defaults, to 0 and `Deltat`, respectively.

- ```
  Format: Binary
  ```

  Optional format of the output data: `Text` (the default), `Binary` (or `Binary64`: 64-bit floating point) or `Binary32` (32-bit floating point), case-insensitive. It must come after `Start` and `Interval` (if they are given) and before `Population`. The `--format` and `--binary` command-line switches override it. See Sec. 3.1.

- ```
  Population: 4.V
  Dendrite: 5
  Propagator: 1 4.phi
  Coupling: 3.nu
  ```

  NFTsim allows the user to specify which objects to output, by entering the appropriate object indices after the labels. For each object, it has some intrinsic fields that will be outputted; for example, `Coupling` outputs `nu`, whereas `CaDP` outputs `nu` and `Ca`.

  For each entry, a field name may be appended after the index with a dot, so that only that field of the object is outputted. If no field name is specified for that entry, then all fields of that object is outputted.

  If a specific field of an object is specified, but that field does not exist, NFTsim checks and returns an error.

3 Analysis

NFTsim produces a single output file, unless a different name is manually specified this file's name

© Brain Dynamics Group, University of Sydney 2015
  ```

is the same as the input file but with the suffix replaced by `.output` . The output file starts with a copy of the input file, to enable the output file to serve as a complete representation of the simulation. The simulation results follow a series of '= ' characters. Example content in `nftsim.output` is

```
==========================================
                  Time             Propagator.2.phi
                                                 0
1.00000000000000e-04     8.87145000000000e+00
```

Each column is a time series with its name indicated in the first line. The first column is always time, and in this example, the second column is `Propagator.2.phi` , indicating that it is ϕ_{ee} (when checked against connection matrix). The node number is indicated in the second line.

It is also worth noting that traces will be written in the order that they are specified. For example, if you write `Population: 3 1` then the columns in the output file will be arranged in this order.

3.1 Binary output

Writing every node at every time-step as text can take most of a simulation's run time, and gives very large files: each value takes 21 or 22 characters. The binary formats (`--binary` or `--format Binary32` on the command line, or `Format:` in the `Output` section) write each value as 8 (or 4) bytes, with no conversion to text. For a 4096-node model writing three traces at every step, this made the run 25 times faster and the file 2.7 times smaller (5.5 times with `Binary32`). The Python and Matlab readers (`utils/nftsim.py` and `nf.read`) detect the format and return the same structures for both, so code that reads text output needs no change.

A binary file starts exactly like a text one: the copy of the configuration file, the line of '= ' characters, a blank line, the row of trace names and the row of node indices. One line of text then follows, of space-separated `key=value` pairs, for example

```
NFTSIM-BINARY version=1 dtype=float64 byteorder=little
   columns=12289 deltat=1.0000000000000000e-04
   interval=1.0000000000000000e-04 rows=2500
```

and the rest of the file is the data, one row per output time, each row `columns` values: the time and then the traces, in the same order as the columns of a text file.

`version` Format version, `1` . A reader should refuse versions it does not know.

`dtype` `float64` (IEEE 754 double precision, the values held by `NFTsim` , unrounded) or `float32` (each value rounded to single precision, about 7 significant digits; note this rounds the time too).

`byteorder` Always `little` : values are little-endian on every platform.

`columns` Number of values per row, including the time; the same as the number of names in the header row.

© Brain Dynamics Group, University of Sydney 2015

`deltat` , `interval` The time-step and the output interval in seconds, with 17 significant digits.

`rows` Number of rows. `NFTsim` writes `unknown` (padded with spaces) when it starts, and the count when the run finishes. So `unknown` means the run was interrupted, or is still running, and the file holds as many complete rows as have been written. When the output cannot be rewritten (written to standard output with `-v` , or to a pipe), it is `stream` .

The line is padded with spaces so that the data start at a multiple of 8 bytes from the start of the file (except for output to standard output), and it ends with a newline (byte 10), immediately after which the data start. Rows are written as the simulation runs, so a file can be read while `NFTsim` is still running, and if a run is stopped, the file holds every row written up to then. The readers keep the complete rows, drop an incomplete last row, and warn when `rows` is not a count or does not match the file (the text reader likewise drops a last line that has no newline).

Text output has 15 significant digits, so values read from it differ from the binary ones by up to about 5×10^{-15} relative; the binary `float64` values are those `NFTsim` computed, bit for bit. The format needs no library to read. In Python, with numpy, the rows can be read with

```
with open('example.output', 'rb') as f:
    while not f.readline().startswith(b'NFTSIM-BINARY'):
        pass
    data = np.fromfile(f, dtype='<f8').reshape(-1, ncols)
```

(use `'<f4'` for `float32` , and `ncols` from `columns` ; for an interrupted run, keep only the whole rows), or simply with `nftsim.NF('example.output')` . In Matlab, after reading the header lines with `fgetl` , `fread(fid, [ncols, Inf], 'double', 0, 'ieee-le')'` reads the rows; `nf.read` does this for you.

3.2 Matlab

A number of Matlab functions are provided to make it easy to manipulate `NFTsim` data from within Matlab. The functions are generally self-documenting with comments at the start of the file. To use these functions, you should add the `NFTsim` root directory to your Matlab path. All of the programs in the `+nf` directory will then be available for use within Matlab as part of the `nf` package (e.g., typing `nf.run()` will execute the function specified in `+nf/run.m`).

Essentially, an output file from `NFTsim` is read into a `nf` struct object in Matlab which simply contains all of the output from `NFTsim` in memory for easy access. Here is an example of a `nf` object:

```
obj =
     fields: {'Propagator.1.phi'  'Propagator.3.phi'}
      nodes: {[1]  [1]}
       data: {[300000x1 double]  [300000x1 double]}
       time: [300000x1 double]
     deltat: 1.0000e-04
    npoints: 300000
```

© Brain Dynamics Group, University of Sydney 2015

- `fields` stores a record of which traces from `NFTsim` are present in the output file
- `nodes` is a cell with the same size as `fields` , and records for each field present, the number of the node in the output.
- `data` is a matrix storing the actual values of the traces.
- `time` is a vector of time values, so that you can plot any of the data traces directly again `obj.time` .
- `deltat` stores the temporal sampling rate.
- `npoints` stores the total number of points in the output.

There are two ways to create the `nf` object. You can read the output file directly after executing `NFTsim` elsewhere

```
obj = nf.read('nftsim.output')
```

or you can use the `nf.run` helper script to run the config file using `NFTsim` and automatically parse the output

```
obj = nf.run('nftsim.conf')
```

`nf.run` writes binary output by default (Sec. 3.1), which is faster to write and read than text and keeps every value exact, unless the configuration's `Output` section sets its own `Format` , which is then used. To choose the format explicitly, pass it as the fourth argument, for example `nf.run('nftsim.conf', false, [], 'Text')` for a human-readable `.output` file. The Python `run` function in `utils/nftsim.py` behaves the same way, with an `output_format` argument.

Several helper files are provided to manipulate the `nf` object. The two most important helpers are `nf.extract` and `nf.grid` . Often you want to extract a particular field from the `nf` object, for example, to examine the output from `Propagator.3.phi` . To do this directly with the nftsim struct, you would need to check the `fields` variable to find the index of the trace you wanted, and then extract it from the `data` field. In the previous example, `Propagator.3.phi` is the second trace. These expressions are identical:

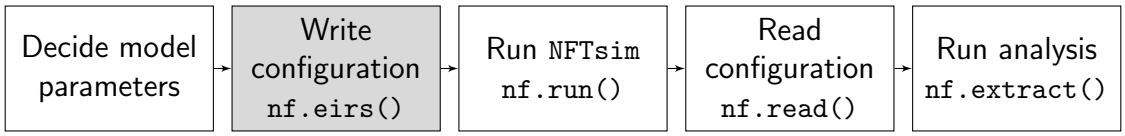


Figure 4: Overview of typical `NFTsim` workflow, showing the roles of the various Matlab tools. For each step, the corresponding Matlab function is shown. Note that the configuration file contains different populations and couplings for different models of neural networks. The Matlab helper `nf.eirs()` is specific to the Robinson et. al. corticothalamic model, and is provided in the `cortiothalamic-model` software package (together with a number of other analysis routines specific to that model) available from the Brain Dynamics website.

© Brain Dynamics Group, University of Sydney 2015

```
trace = obj.data{2};
trace = nf.extract(obj,'Propagator.3.phi')
```

`nf.extract` quickly becomes useful when there are many different fields. It is not case sensitive (so `propagator.3.phi` works as well). You can also specify using additional arguments to extract only a portion of the time series, and also to select a subset of nodes. Finally, you can also provide multiple traces to concatenate them into a single matrix. For example,

```
trace = nf.extract(obj,'propagator.1.phi,propagator.3.phi');
```

will create a 300000×2 matrix with both of the traces.

Finally, if you run `NFTsim` with multiple nodes, it typically solves the system of equations on a square grid. Therefore, if you have output for notes 1-400, this corresponds to a 20×20 grid. `nf.grid` allows you to request a trace from the `nf` object and have it reshaped into a square grid. This lets you easily make surface plots of the data, or perform tasks that are spatially dependent.

One important task is computing the power spectrum as predicted by the linearized analytic equations. This can be achieved using `nf.spatial_spectrum` which takes in an `nf` object and computes the power spectrum integrated over k taking into account volume conduction.

4 Development

This section provides programming information about `NFTsim` for development extension and core logic. This section assumes a basic knowledge of `ANSI C++` , including object oriented programming, usage of template, and standard template library (STL). As our development takes place within a Git repository, a working knowledge of Git will significantly help.

A solid command of object-oriented programming is an essential prerequisite. Given the inevitable complexity arising from many developers and class relationships, good code structure and object-oriented programming principles must be adhered to whenever reasonable. One of the primary aims of object-oriented program as used by `NFTsim` is code reuse. Duplicated code, with subtle variations between them, is one of the best ways to introduce undetected numerical errors. The ultimate consequence is erroneous publication and refactoring of the code. Code reuse is one of the highest priorities. Sec. 4.1 contains methods for code reuse. When in doubt, read existing class implementations for examples, or contact the developers for assistance (`braindynamics@physics.usyd.edu.au`).

Here is a list of training resources:

- Cpp resource
- Git resource
- OOP resource

© Brain Dynamics Group, University of Sydney 2015

4.0.1 Development workflow

We have adopted the *fork and pull* development model using GitHub. If you would like to contribute to `NFTsim` , set up your repository as follows:

1. Make a private fork of the main repository.
2. Clone your fork.
3. Set an upstream remote pointing at the main `NFTsim` repository.
4. Use `git pull upstream master` to merge changes from the main repository into the master branch of your working copy.
5. We recommend using a branch in your fork for developing modifications
6. When your feature is ready for integration into the main repository, submit a pull request on GitHub. Developers within the Brain Dynamics Group will then be able to review your code, check compatibility, and perform the merge.

Note that you do not have write access to the main repository, so all changes must be proposed via pull requests. If your proposed changes are accepted, we may make modifications to ensure they integrate seamlessly with the rest of the program.

As discussed at the start of manual, `NFTsim` encapsulates each component of the model with an object: `Coupling` , `Dendrite` , `FiringResponse` , `Propagator` . The `Dendrite` and `FiringResponse` (and also `Timeseries`) are contained within `Population` .

Each object class may be overloaded for more sophisticated behaviour; for example, `Propagator` may be overloaded to perform wave propagation, or `Coupling` may be overloaded for synaptic plasticity.

4.0.2 Coding style

To maintain consistency within the `NFTsim` code base, we strongly encourage you to adhere to these programming conventions used throughout the project:

Tabs:	two spaces.
Braces:	the K&R style.
Class names:	UpperCamelCase
Function names:	lowerCamelCase
Variable names:	lowercase

When using the `C++` standard library, use the `using` directive to explicitly indicate the components you are using. For example,

```
#include<vector>
using std::vector;
```

© Brain Dynamics Group, University of Sydney 2015

4.1 Extending NFTsim via inheritance

Most new functionalities may be introduced by inheriting existing classes and overloading appropriate functions, where the core classes are:

Class	is Responsible for	Field
`Timeseries`	A function of time, predominantly used as stimulus.	
`Dendrite`	Dendritic response.	V_{ab}
`FiringResponse`	Firing response of population.	V_a
`Population`	Contains `Timeseries` , `Dendrite` , and `FiringResponse` .	Q_a
`Propagator`	Axonal firing propagation	ϕ_{ab}
`Tau`	Axonal propagation time latency	τ_{ab}
`Coupling`	Synaptic coupling	ν_{ab}

Examples where these classes are inherited to provided new functionalities include:

Derived class	Base class	Extension
`Wave`	`Propagator`	Wave propagation.
`CaDP`	`Coupling`	Plastic synapse.
`LongCoupling`	`Coupling`	Nonlocal synapse.

4.1.1 Class hierarchy

The diagram in Figure 4.1.1 shows the inheritance hierarchy for the base NFTsim objects. Note that the `Array` class is a container object, and it is not necessary to interact with it directly. See Sec. 4.6.2 for details.

4.1.2 Procedure for implementing a new class

Implementing a new class may be done using the following procedure:

1. Identifying the core class to inherit (see above tables). Then look up the documentation of the appropriate base class from Sec. 4.2.2–4.2.7.
2. Decide on the name of the class. Generally, it may be advantageous for the new name to refer to the base name, plus a terse description. For example, `LongCoupling` refers to its base class `Coupling` , with the additional description indicating that the new class has long range coupling. The file names should be the same as the class name, but all in lower case, for consistency.
3. Overload appropriately the `init()` (see Sec. 4.3), `output()` (see Sec. 4.4), and `step()` functions. **Do not** copy and paste code. Rather, use

```
BaseClass::function();
```

© Brain Dynamics Group, University of Sydney 2015

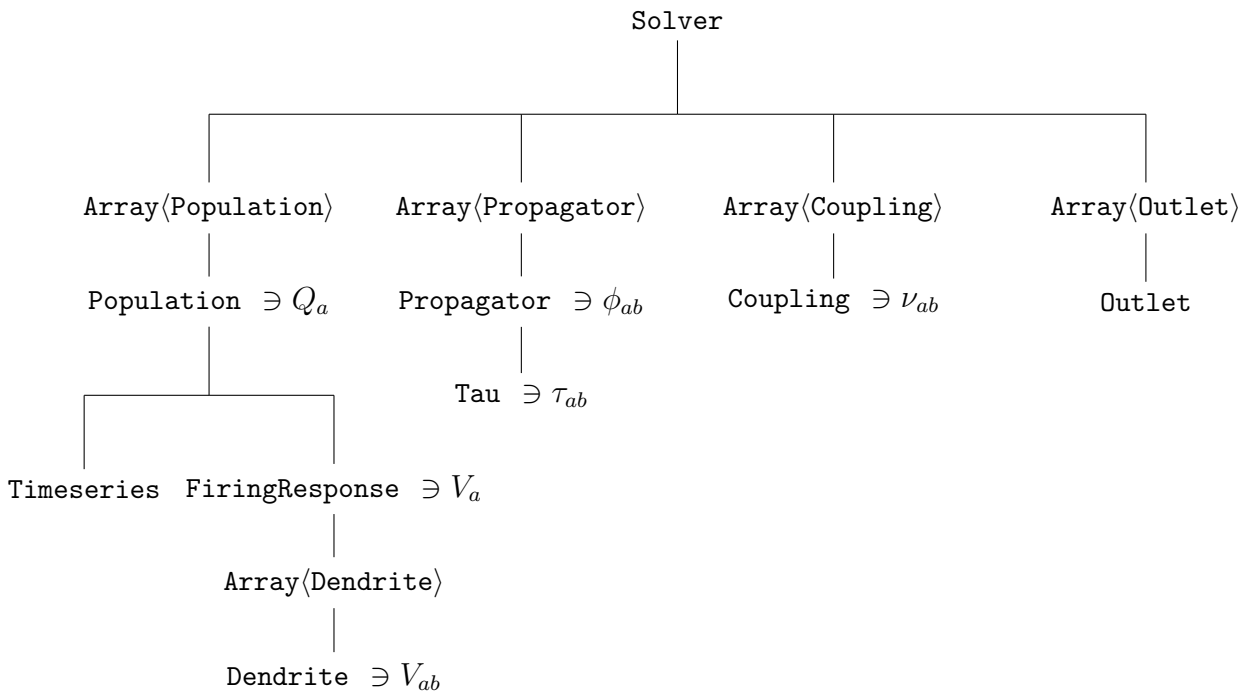


Figure 5: Schematic of the main class structures in `NFTsim` . Each line indicates that the bottom class is a member of the top class. The `a` $\ni b$ symbol indicates that the dynamical field $b(\mathbf{r}, t)$ is a member of the class `a` . Inheritance structures are NOT illustrated.

which will utilize the existing function in the base class. This will ensure that your derived class is updated if the base class changes.

4. It is likely that differential equations are solved in the new class. `NFTsim` provides two classes, `DE` and `Stencil` , that solves spatially homogeneous differential equations, and spatially inhomogeneous equations, respectively. See Sec. 4.5.

5. Register the new class as documented in Sec. 4.2.2–4.2.7.

6. Write a configuration file that uses the new class. Or if the object may exhibit different types of behaviour under different parameter values, having one configuration file for each type of behaviour may be advantageous. Make sure that the configuration file has an appropriate comment.

4.2 NFTsim classes

This section contains an overview of all of the base classes in NFTsim.

4.2.1 Class NF

All core classes are derived from the `NF` object. This abstract base class contains 3 member variables, and 3 interface methods:

© Brain Dynamics Group, University of Sydney 2015

Variable	
`nodes`	The number of nodes as specified from the configuration file.
`deltat`	The time increment per timestep in units of seconds.
`index`	The index associated with the object.
Methods	
`init(Configf& configf)`	Initializes the object with the config file.
`step(void)`	At each timestep, this function is called.
`output(Output& output) const`	Specifies which fields to output.

All `NF` classes automatically handles the `ofstream::<<` and `ifstream::>>` operators.

When appropriate, the default constructor, copy constructor, and `operator=` should be made inaccessible by declaring them to be private.

4.2.2 Population

Models a neural population, which may be either a stimulus or normal population. If it has any `Dendrites` , i.e. it has presynaptic connections, then it is a normal population, and it is a stimulus if it does not have `Dendrites` .

In the former case, it contains the `FiringResponse` class and have a soma potential; in the latter case it contains the `Timeseries` class, and does not have a soma potential.

In either cases, the `Population` class has a keyring storing the firing rate history, coded as a 2D array plus an integer key. The history holds $D_a + 1$ time-points, where D_a is the largest delay (in time-steps) of any propagator leaving the population: a delay of D_a steps needs the current value plus D_a past values. (The Time delays section of Sanz-Leon et al., 2018, gives the depth as D_a; versions of `NFTsim` before this correction used that depth, which turned a delay equal to the history length into no delay.)

Functions `sheetlength()` and `glu()` provides access to the physical length and glutamate concentration in synaptic cleft, respectively.

A population is "settled" after `Population::init()` is called, after which no dendrites can be added. The firing rate history is grown afterwards, as each `Propagator` reads its `Tau` .

4.2.3 Propagator

The `Propagator` class implements the axonal propagation as an identity map, i.e.

$$\mathcal{D}_{ab} = 1.$$

To introduce more sophisticated axonal propagation, this class is inherited and overloaded.

© Brain Dynamics Group, University of Sydney 2015

The `Propagator` class provides a constant references to both presynaptic and postsynaptic populations.

Object `Tau` provides the axonal delay latency of propagator.

For spatially inhomogeneous propagators, class `Stencil` provides a Moore grid, as documented in Sec. 4.5.2.

To "register" your propagator, look for the

```
// PUT YOUR PROPAGATORS HERE
```

subsection in `solver.cpp` .

4.2.4 Coupling

The `Coupling` class manages ν_{ab}, which is constant in space and time.

To introduce synaptic plasticity, derive from this class.

The `Coupling` class provides a constant references to both presynaptic and postsynaptic populations. Glutamate concentration is also provided.

Function `excite()` indicates whether this is an excitatory coupling. Protected variable `pos` is $+1$ or -1, depending on the sign of ν_{ab}.

To "register" your coupling, look for the

```
// PUT YOUR COUPLINGS HERE
```

subsection in `solver.cpp` .

4.2.5 FiringResponse

To implement new firing response dynamics, inherit from class `FiringResponse` , where `init()` and `fire()` should be overloaded.

Object `dendrites` is an array of presynaptic `Dendrite` .

Objects `glu_m` and `glu_rk4` calculates glutamate concentration in the synaptic cleft, which is accessed via `glu()` .

To "register" your firing response, look for the

```
// PUT YOUR FIRING_RESPONSE HERE
```

© Brain Dynamics Group, University of Sydney 2015

subsection in `population.cpp` .

4.2.6 Stimulus

To implement new stimulus pattern, inherit from class `Timeseries` , where `init()` and `fire()` should be overloaded.

The time is provided via the variable `t` .

To "register" your stimulus, look for the

```
// PUT YOUR TIMEFUNCTION HERE
```

subsection in `timeseries.cpp` .

4.2.7 Dendrite

To implement new dendritic responses, inherit from class `Dendrite` .

References are provided for the presynaptic coupling and propagator.

To "register" your dendritic response, look for the

```
// PUT YOUR DENDRITE HERE
```

subsection in `firing_response.cpp` .

4.3 Adding variables to the configuration file

Your new class might require additional parameters in the configuration file. Input via the configuration file is implemented in the `init()` function, via `Configf` , which provides the following functions:

1. `param()` : go to the next keyword and reads in a variable. If the keyword is not found, barks and exits.
2. `optional()` : same as param, but does not bark nor exit. The use of this function is discouraged, since it may introduce subtle bugs or human errors.
3. `numbers()` : reads an arbitrary number of white-space separated numbers, returned in a `vector` .

© Brain Dynamics Group, University of Sydney 2015

Sometimes the config file may search for either one parameter, OR another one. For example, `Wave` propagator accepts parameter `gamma` or `velocity` such that `gamma` = `velocity` /`range` , so that we may have either of these:

```
Wave - Range: 80e-3 gamma: 116
```

```
Wave - Range: 80e-3 velocity: 9.28
```

Then, we may use this pattern with `optional()` to achieve the desired effect:

```
configf.param("Range",range);
if( !configf.optional("gamma",gamma) ) {
  double temp; configf.param("velocity",temp);
  gamma = temp/range;
}
```

4.4 Adding variables to the output file

`NFTsim` outputs field variables at every timestep, where the user chooses which object to output via the configuration file, and the object chooses which fields to output in the `output()` function.

To output field solutions, overload `NF::output()` to write

```
output.prefix("Object␣Name",index+1);
output("field1",field1);
output("field2",field2);
subobject1.output(output);
subobject2.output(output);
BaseClass::output(output);
```

or for a single output field,

```
output("Object␣Name",index+1,"field1",field1);
```

where `field1` , and `field2` are `vector<double>` with size equal to the number of spatial nodes.

To output a single number, as opposed to a spatial field, use the function

```
output.singleNode("Object␣Name",index+1,"field1",field1);
```

or

```
output.singleNode("field1",field1);
```

where `field1` is a `vector<double>` of size 1.

© Brain Dynamics Group, University of Sydney 2015

4.5 Tools for solving differential equations

Classes `DE` and `Integrator` (currently RK4 is implemented) are used to solve generic systems of ODEs, where the dynamical variables are homogeneous fields. For spatial dependency, a five-point (von Neumann) stencil is provided in `Stencil` , used by `Wave` ; a nine-point stencil is provided in `StencilLegacy` , used by `WaveLegacy` .

4.5.1 Class DE

Class `DE` and `RK4` together solves ODEs of homogeneous fields.

To define your differential equation, declare a new class that inherits `DE` . Overload the `rhs()` function to define the differential equation. For example, the differential equation

$$F = m\frac{d^2x}{dt^2},$$

is redefined as

$$F = y_0,\ x = y_1,\ \frac{dx}{dt} = y_2,$$

so that it can be formulated as a system of 1st order differential equations

$$\frac{dy_0}{dt} = 0,\ \frac{dy_1}{dt} = y_2,\ \frac{dy_2}{dt} = my_0,$$

with

$$y_0 = F,$$

being an algebraic equation.

This can be defined in `NewDE::rhs()` as

```
void NewDE::rhs( const vector<double>& y, vector<double>& dydt )
{
  // y = { F, x, dxdt }
  dydt[0] = 0;      // F, leave unchanged
  dydt[1] = y[2];   // x
  dydt[2] = m*y_0; // dxdt
}
```

where the comments are strongly recommended for readability.

Declare the number of differential equations (in this example, 3) in the constructor

```
NewDE( int nodes, double deltat ) : DE(nodes,deltat,3) {}
```

To integrate `NewDE` , declare a pair of `DE` and `RK4` objects:

```
NewDE de(nodes,deltat);
RK4 rk4(de);
```

© Brain Dynamics Group, University of Sydney 2015

then the differential equation may be solved via (usually done in `Class::step()`)

```
for( int i=0; i<nodes; i++ )
  de[0][i] = F; // algebraic equation
rk4.step();     // integrate differential equation by one step
```

Often, the `NewDE` class is incorporated in a `NewClass` . In such cases, it is advantageous to declare `NewDE` as `struct` , which is a class where all members are public by default, and have this new `NewDE` declared within `NewClass` , so that it is accessible within it. All parameters of the differential equation (in the example above, m) will then belong to `NewDE` .

Since `NewClass` has access to all members of `NewDE` , all initialization of `NewDE` may be done in `NewClass::NewClass()` and `NewClass::init()` .

Remember to redirect the interface and output variables to members of `NewDE` . For example,

```
vector<double>& NewClass::x(void) const
{
  // original code from base class:
  // return _x;
  // new code in derived class replaces original x variable:
  return (*de)[1];
}
```

```
void NewClass::output( Output& output ) const
{
  output.prefix("NewClass",index+1);
  // original code from base class:
  // output("x",_x);
  // new code in derived class replaces original x variable:
  output( "x",(*de)[1] );
}
```

To extend an existing `DE` class, for example extend `NewDE` to `New2DE` , inherit `New2DE` from `NewDE` . The `extend()` function allows the introduction of new differential equations and field variables.

4.5.2 Stencil

Class `Stencil` provides Moore grid for spatially inhomogeneous calculations.

Given a stencil,

```
Stencil stencil(nodes,longside,"Torus");
```

Use `operator=` to set the spatial field values of a `vector<double>` .

The stencil pointer can be set and get via the `set()` and `get()` functions, and incremented via

© Brain Dynamics Group, University of Sydney 2015

`operator++` . The Moore grid can be read with

```
stencil(nw); stencil(n); stencil(ne);
stencil( w); stencil(c); stencil( e);
stencil(sw); stencil(s); stencil(se);
```

4.6 Core logic

Normal extension of `NFTsim` *does not involve any modification of the core logic. Proposed to the core logic are unlikely to be accepted. This section is intended to provide context to understand the design of the base objects.*

One integration step of the model implements the following stages:

1. Coupling response, $P = \nu\phi$, using ϕ from the previous step.
2. Dendritic response, advancing each V_{ab} one step with that input.
3. Afferent summation.
4. Firing response/stimulus response, giving the new Q.
5. Propagator step (wave equation integration, or a direct map), which includes Q delay processing and uses the new Q.

Most of the computational *load* comes from integrating wave equations and harmonic oscillators within the dendritic responses. Most of the execution *time* is probably spent writing the output file.

Wave equations are integrated by explicit finite differences (Eq. 32 of Sanz-Leon et al., 2018, after Rennie, 2001), using a five-point spatial stencil (`Wave`); `WaveLegacy` uses a nine-point stencil. Other parts of code are unaffected by spatial geometry so this can be switched to irregular gridding easily.

The default `Dendrite` integrates its second-order response with RK4, holding the input $\nu\phi$ constant over each step. The `Integral` dendrite instead uses the exact solution for an input that varies linearly over the step, which is cheaper. Neither is fourth order overall, because of how the input is supplied (see *Accuracy* below).

4.6.1 Accuracy

The `Wave` update on its own is second-order accurate in space and time. However, the modules are advanced in sequence within a step (see *Program flow*): each coupling uses ϕ from the previous step, each dendrite holds (RK4) or extrapolates (`Integral`) that input over the step, and propagators then use the new firing rates. This splitting makes the coupled system *first*

© Brain Dynamics Group, University of Sydney 2015

order in `Deltat` , whatever integrator each module uses. For example, for a deterministic single-node corticothalamic model driven by a 7.8 ms pulse, the maximum error in the excitatory soma potential relative to a $\Delta t = 2^{-19}$ s reference was about 11%, 6%, 3% and 1.4% of the evoked response for $\Delta t = 2^{-13}$, 2^{-14}, 2^{-15} and 2^{-16} s. When fitting transient responses, check that results are converged in `Deltat` , for example by repeating a run with half the time-step.

4.6.2 Class Array

`Array` is a container array to store objects that supports the `ofstream::<<` and `ifstream::>>` operators, as well as a `step(void)` function. This object typically is, but is not required to be, an `NF` object.

The `step(void)` function is equivalent to a `foreach(element).step()` in pseudocode. This function is encouraged over the use of `empty()` , `size()` , and `operator[]()` , which are discouraged to be used.

4.6.3 Program flow

Essentially, the program flow can be read from Fig. 4.1.1, so that objects take priority from top to bottom, left to right, both in terms of initialization and stepping through each timestep. A more detailed description is given below, and the reader is referred to the source code for complete description.

We use the semicolon to denote a succession of functions/procedures, and `a()` $\Rightarrow$ `b()` symbol to denote function `b()` as content of function `a()` .

`main()`	$\Rightarrow$	Initialize the config file, dump file and output file; `Solver::init()` ;`Solver::solve()` ;
`Solver::init()`	$\Rightarrow$	read in global parameters; Read in `CntMat` ; Construct `Population` ; construct `Propagator` ; construct `Coupling` ; `Population::add2Dendrite()` ; Read configurations for `Population` , `Propagator` , `Coupling` , and `Output` .
`Solver::solve()`	$\Rightarrow$	`for(...)` { `Solver::step(); Output::step()` ; }
`Solver::step()`	$\Rightarrow$	`Coupling::step()` ; `Population::step()` ; `Propagator::step()` ;
`Population::step()`	$\Rightarrow$	`FiringResponse::step()` if neural population; `Timeseries::step()` if stimulus
`FiringResponse::step()`	$\Rightarrow$	`Dendrite::step()` ; sum over V_{ab}

© Brain Dynamics Group, University of Sydney 2015

4.7 Output routine

To accommodate the coding interface for `NF::output` , the output routine of `NFTsim` involves 4 separate classes: `Outlet` , `Output` , `Outputs` and `Dumpf` .

Class	Role
`Outlet`	Stores a reference to field variable (`vector<double>`) and its associated name.
`Output`	Helper class in the parsing of which objects and which fields to output, according to the configuration file.
`Outputs`	Contains `Array<Outlet>` and performs output routine.
`Dumpf`	File handle (maybe to `stdout`) to output. It writes text, or, in the binary formats, the metadata line (`binaryHeader`), each row (`binaryRow`) and finally the number of rows (`finish`).

4.8 About object-oriented programming

Working knowledge of object-oriented programming (and `C++`) is an essential prerequisite for developing `NFTsim` . While object-oriented programming is standard computing knowledge and material and references on the topic is abundant, here we outline the motivation behind object-oriented programming, and in particular the reason for using it on `NFTsim` .

4.8.1 Procedural programming

1. *Procedural programming* consists of *variables* and *functions*. Variables (and structs) are the "nouns"; functions are the "verbs."

2. In procedural programming, there are no inherent *code structure* in the code that is enforced by the language. Rather, code structure is given by, for example, file systems.

4.8.2 Object-oriented programming

1. *Object oriented programming* generalizes variables and structs to *objects*, where an object is a collection of variables and functions.

2. While an object is a noun, it is capable of performing actions and having other objects perform actions on them.

3. The idea of an object performing actions and being the recipient of actions lead to the idea of the *interface* of an object.

4. This separation between implementation (the actual algorithmic code) and interface gives rise to encapsulation, inheritance and polymorphism:

   Encapsulation is the *hiding of the implementation* that is not part of the immediate interface.

© Brain Dynamics Group, University of Sydney 2015

Inheritance is where a class of objects *reuses* and *extends* the implementation and/or interface of a more fundamental class.

Polymorphism is where objects of the same interface perform their own appropriate actions. This gives rise to dynamic object types and behaviour.

5. To summarize, some features brought forth are code structure and protection, code reuse and extension, dynamic allocation of object type, and (if applicable) coding-level object hierarchy.

4.8.3 NFTsim

1. `NFTsim` is naturally object oriented: firing response, propagators, couplings, dendrites naturally arise and form a network, where each class/object influences other ones.

2. The extension of simple objects to more sophisticated ones (e.g. from static to plastic synaptic coupling) may naturally be done via inheritance. In many cases, the interface is kept while the implementation is extended.

3. The object type is determined during *runtime*, while reading the config file. Thus, *polymorphism* comes into play.

4. All fundamental classes in `NFTsim` has already been inherited. Among the inherited classes, many have fundamental relationships with classes in the inheritance hierarchy, e.g. `Wave` propagator may degenerate to `Harmonic` , `BCM` coupling extends `CaDP` .

5. Given these complexities, which will only increase in the future, object-oriented programming principles MUST be adhered to whenever reasonable. Failure to do so will inevitably lead to unmaintainable (and unacceptable) code that may well introduce undetected numerical error.

© Brain Dynamics Group, University of Sydney 2015